SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	**CO4:** Articulation (BL4, BL5)	**Assessment:**	Assessment Tool 3 – Comparative Analysis
Weightage:	10% of CIA	**Total Marks:**	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	S.SRI MATHI	**Reg. No.:**	192424095
Section / Batch:	SLOT B	**Date:**	24-08-2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Analyze the given questions thoroughly before answering.
- Compare multiple aspects such as methods, results, advantages, and limitations clearly.
- Critically evaluate the strengths, weaknesses, and implications with proper justification.
- Present meaningful insights, interpretations, and well-supported conclusions from the comparisons.

Question Type	Focus Area	Questions
Comparative Analysis	Comparative analysis of Dynamic Programming and Greedy algorithms for optimization problems.	Q1

SECTION A – Comparative Analysis

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Comparative Analysis					
Criteria	**Weightage**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Selection of Studies/Parameters	20%	Selects highly relevant and diverse studies with well-defined comparison parameters	Selects relevant studies with minor gaps in parameter definition	Limited or partially relevant selection	Poor or inappropriate selection of studies
Comparison & Differentiation	25%	Clearly compares multiple aspects (methods, results, limitations) with strong differentiation	Good comparison with minor gaps	Basic comparison with limited depth	No clear comparison; mostly descriptive
Critical Evaluation	25%	Critically evaluates strengths, weaknesses, and implications with strong justification	Provides evaluation with some justification	Limited evaluation; lacks depth	No critical evaluation provided
Synthesis & Insight Generation	20%	Generates meaningful insights and conclusions from comparisons	Some insights derived from comparison	Limited or unclear insights	No insights generated
Clarity	10%	Information is clearly presented (tables/figures) with logical structure	Generally clear with minor issues	Presentation lacks clarity or structure	Poor presentation and difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Selection of Studies/Parameters	20%				
Comparison & Differentiation	25%				
Critical Evaluation	25%				

Synthesis & Insight Generation	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

ASSESSMENT TOOL 3 – Comparative Analysis

NAME: **S.SRI MATHI**

REG.NO: **192424095**

COURSE CODE: **CSA0603**

COURSE NAME: **DESIGN AND ANALYSIS OF ALGORITHM**

DATE: **24.08.2026**

1. Real-Time Industry-Based Scenario

Modern digital platforms process large amounts of textual data. **Document editors, e-book platforms, web applications, and messaging systems** need to arrange words efficiently within a given display width. This represents the **Word Wrap problem**. At the same time, sequence-processing applications analyze strings to identify structural patterns. The **Longest Palindromic Subsequence (LPS)** problem represents this type of optimization.

For example, consider the text:

"Artificial intelligence is transforming modern business through automation."

A document-layout system must determine suitable line breaks while minimizing unnecessary spacing. In contrast, an LPS system examines the characters of a string and identifies the longest subsequence that remains identical when read in reverse.

Thus, both problems use Dynamic Programming, but they optimize **different properties of textual data**.

2. Problem Definition and Objective

Aspect	Word Wrap	Longest Palindromic Subsequence
Problem	Divide words into suitable lines	Find the longest palindromic subsequence
Objective	Minimize line-spacing penalty	Maximize palindrome length
Input	Sequence of words	Sequence of characters
Output	Optimized line arrangement	Longest palindromic subsequence

Optimization	Minimization	Maximization

For Word Wrap, a common objective is:

$$\textbf{Minimize Total Cost} = \sum (\textbf{ExtraSpace})^2$$

For LPS:

$$\textbf{Maximize } |P|$$

where P represents the palindromic subsequence.

3. Dynamic Programming Approach

Both problems have **optimal substructure and overlapping subproblems**, making Dynamic Programming suitable.

Word Wrap

Let:

$$dp[i] = \text{minimum cost for arranging the first } i \text{ words}$$

The recurrence is:

$$dp[i] = \min_{j<i} \{dp[j] + Cost(j,i)\}$$

The algorithm considers possible positions for the previous line break and selects the arrangement with the minimum total cost.

LPS

Let:

$$dp[i][j] = \text{length of the LPS between positions } i \text{ and } j$$

If the two characters match:

$$dp[i][j] = dp[i+1][j-1] + 2$$

Otherwise:

$$dp[i][j] = \max(dp[i+1][j], dp[i][j-1])$$

Therefore, Word Wrap uses DP for **optimal partitioning**, while LPS uses DP for **optimal subsequence selection**.

4. Algorithm and Steps

Word Wrap	LPS
1. Store the length of each word.	1. Take the input string.
2. Define the maximum line width.	2. Create a two-dimensional DP table.
3. Determine which consecutive words can fit on each line.	3. Initialize every single character with value 1.
4. Calculate the unused space and corresponding penalty.	4. Compare the characters at the two ends of each substring.
5. Store the minimum cost using Dynamic Programming.	5. If they are equal, include both characters.
6. Backtrack through the DP results to obtain the optimal line breaks.	6. If they are different, consider excluding either character and select the larger result.

Example

Word Wrap

Consider:

```
Artificial intelligence is transforming modern business
```

with a maximum line width of 20.

A possible optimized arrangement is:

```
Artificial
intelligence is
transforming modern
business
```

The DP algorithm evaluates different combinations and selects the arrangement with the lowest spacing penalty.

LPS

Consider:

```
character
```

The longest palindromic subsequence is:

```
carac
```

Therefore:

LPS Length=5

The characters do not have to be consecutive because LPS considers a **subsequence**, not a substring.

5.Recurrence Comparison

Word Wrap	LPS
dp[i]=min(dp[j]+Cost(j,i))	If S[i]=S[j]: dp[i][j]=dp[i+1][j−1]+2
Uses min()	Uses max()
Prefix-based DP	Interval-based DP
Minimizes total penalty	Maximizes subsequence length

The comparison demonstrates that the **objective function directly influences the recurrence relation**.

6. Correctness Analysis

Word Wrap

The algorithm considers every valid position at which the previous line could end. For each possibility, it combines the previously calculated optimal cost with the cost of the current line. Selecting the minimum guarantees the optimal arrangement under the defined cost function.

LPS

For every substring, the algorithm considers its two boundary characters. When they match, they can contribute to the palindrome. When they do not match, both possibilities of excluding one boundary are considered. Taking the maximum therefore produces the longest possible palindromic subsequence.

7.Complexity and Scalability

Measure	Word Wrap	LPS
Time Complexity	**$O(n^2)$**	**$O(n^2)$**

Space Complexity	**O(n)**	**$O(n^2)$**
DP Structure	1D	2D
Scalability	Comparatively better	More memory-intensive

Both algorithms have quadratic time complexity. However, their space requirements differ considerably.

For example, if:

$$n=10,000$$

an LPS implementation using a two-dimensional table may require:

$$10,000\times10,000=100,000,000$$

DP entries.

Therefore, although both algorithms have the same asymptotic time complexity, **LPS becomes more challenging to scale because of its quadratic memory requirement**.

8. Real-Time Industry Comparison

Word Wrap

The Word Wrap concept is applicable to:

- Digital document editors
- E-book readers
- Web applications
- Messaging applications
- Digital publishing systems

Real-world text-layout systems are more advanced than the basic DP model because they also consider font metrics, Unicode characters, justification, hyphenation, formatting, and page constraints.

LPS

LPS represents a fundamental sequence-processing technique applicable to:

- String analysis
- Pattern identification
- Sequence processing
- Computational biology and bioinformatics research

However, real-world biological and textual datasets can be extremely large. Therefore, specialized algorithms and memory-efficient approaches may be required instead of directly constructing a complete O(n2) table.

9. Advantages and Limitations

Word Wrap

- Produces an optimized line arrangement.
- Requires comparatively less DP memory.
- Useful for text-layout optimization.
- Basic model does not represent all real formatting constraints.

LPS

- Produces the longest palindromic subsequence.
- Provides an optimal sequence result.
- Useful for sequence-pattern analysis.
- Quadratic memory limits scalability for very long strings.
- Basic DP becomes expensive for large sequences.

10. Comparative Insights

1. Different optimization objectives

Word Wrap is a **minimization problem**, whereas LPS is a **maximization problem**.

2. Different DP structures

Word Wrap uses a **prefix-based 1D DP**, while LPS uses an **interval-based 2D DP**.

3. Different outputs

Word Wrap produces a **layout structure**, whereas LPS produces a **sequence structure**.

11. Conclusion

Word Wrap and Longest Palindromic Subsequence demonstrate two distinct applications of Dynamic Programming in text processing.

Word Wrap focuses on optimizing the **presentation of text** by selecting appropriate line breaks and minimizing spacing penalties. **LPS** focuses on optimizing the **structure of a string** by finding the longest subsequence that forms a palindrome.

Although both use Dynamic Programming and have O(n2) time complexity, they differ in their objectives, state representations, recurrence relations, outputs, and memory requirements.

Word Wrap → Minimize Layout Cost LPS → Maximize Palindromic Length

The comparison establishes that Dynamic Programming is a general optimization methodology rather than a single fixed algorithm. Its effectiveness depends on correctly identifying the **state, recurrence, objective function, and overlapping subproblems** of the given application.